

YoSOM

A Two-Sided AI System for Yale SOM Dining

Aligning supply with student demand at Charley's Place and McNay Cafe

MGT 575 — Generative AI for Business

Final Project Report

Guo Yu, Hsin-Yuan Lee, Junyi Tang, Amy Qiu, Linhang Ren, Xiaoxiao Wu

Yale School of Management

May 4, 2026

Executive Summary

Yale SOM dining runs on guesswork. Charley's Place and McNay Cafe plan their menus, prep their kitchens, and manage their inventory within the blackbox. In turn, students feel like the menu just appears in front of them. Students have no real channel to say what they want and speak up, and how they can engage and shape the SOM community together.

YoSOM is our attempt to close that gap. It is a two-sided generative AI system. On one side, students open a mobile-first chatbot via QR code or link, share what they want to eat next week, vote on candidate ideas, and flag dietary needs, all in under a minute. On the other side, catering managers open a vendor-intelligence dashboard that turns those raw inputs into a daily ops brief: what to add, keep, or drop from next week's menu, an event-aware demand forecast, an estimated dollar impact, and a one-line voice-of-student headline. It also generates promotional posters for next-week menu features that vendors can post back into the student app or SOM media channel.

Working app: <https://yosom.vercel.app> (student) and <https://yosom.vercel.app/manager> (vendor). Source code: <https://github.com/yuanlee218/YoSOM>. The repository includes a README, the manager-side standard operating procedure, dummy seed data, and a one-line setup guide, so a new reader can run the demo locally in under five minutes.

1. Introduction

1.1 Problem and Why this matters

Yale SOM has roughly 700 students who eat on campus most days. Charley's Place and McNay Cafe are the two main dining venues, and between them they serve thousands of meals a week. The economics are not trivial: dining services run on tight margins, food waste is a real cost, and student satisfaction is a soft-but-very-visible measure of how well the SOM is taking care of its community. Today, the venues plan menus largely on tradition and gut feel. There is no structured way for a student to say "I'd actually eat butter chicken twice a week if you'd put it on the menu," and there is no structured way for a manager to know that 12 students said exactly that this week.

Two specific pain points motivated us. First, a noticeable share of food prepared at peak hours gets tossed by closing time: anyone who has walked past Charley's at 2 PM has seen it. Second, when surveys do happen, they happen rarely and feel performative; students fill them out (or don't) and never see a result. The status quo costs the school money, costs students food they actually want to eat, and quietly erodes trust between students and the SOM operation team.

We picked this problem for three aspects: it is a real operational problem with a measurable downside, it has a natural conversational interface (telling someone what you want to eat is easy and fun in a way that

filling out a survey is not), and the underlying data: preferences, votes, free-text quotes, is exactly the kind of unstructured input that modern LLMs are good at turning into structured action.

1.2 Who it serves

There are two distinct user types, and the product is shaped around the information asymmetry between them.

1. **Students and faculty.** They are the supply of data input. The student-facing experience is a chat: choose a venue, tap through preference chips, vote on four candidate items, optionally type whatever else is on their mind, and see a live feed of what other students are voting for.
2. **Venue managers and dining staff.** They are the consumers of data input. We assume they have a dedicated time before the week starts to triage food decisions, and want a clear answer to "what should I prep more of, less of, or differently next week?" Therefore, the manager-facing experience is a desktop dashboard with a sidebar: Overview, Demand Forecast, AI Ops Brief, Raw Data, and Posters.

1.3 What the system does

Inputs and outputs in plain language. Input: a student opens the app, picks Charley's or McNay, and is asked: which flavors, which proteins, which cuisines, which dietary tags? Then they vote on four candidate menu items and (optionally) type a more detailed answer to "what are you actually craving?" Each interaction writes a typed document to MongoDB: a Vote, a Preference, a Reaction, or a DeepTalk transcript with extracted tags.

Output: On the manager's side, those documents get aggregated and synthesized. The Overview page shows live KPIs (sessions, satisfaction pulse, predicted servings, free-text count) and several charts. The Forecast page takes a baseline daily-servings number from our cost table and applies three multipliers: a survey-signal multiplier from the net direction of votes, an event multiplier from the campus-events calendar (free-lunch days, ceremonies, networking receptions), and an exchange multiplier from the SOM students-away roster. Multiplied together over seven days, that becomes a forecast with confidence bands. The AI Ops Brief is where Claude actually composes prose: given the aggregated signals, the forecast, the cost table, and the event/exchange context, it returns a structured JSON brief with menu add/keep/drop recommendations, a forecast summary, a budget impact line, dietary flags, top quotes, and a one-line voice-of-student headline. The Raw Data page is the receipt. The Posters page calls OpenAI's image API to generate promotional visuals that can be pushed back into the student app.

1.4 The generative-AI angle

Three places where we used generative AI rather than rules or simpler statistics, with the rationale for each:

- **Claude Haiku 4.5 — student conversation glue and tag extraction.** A regex of food keywords could classify on-topic vs off-topic input, but it would feel mechanical and would miss real signal. A small LLM does this naturally and cheaply, and the post-session tag-extraction step turns every chat into a structured set of preference tags that downstream analytics can use.
- **Claude Haiku 4.5 — ops brief generation.** The brief is the place where the system has to do real synthesis: turn 50 votes plus 12 quotes plus a forecast into three actionable recommendations a busy manager will read in a minute. This is the kind of work that is hard to specify with rules. We started with Claude Sonnet 4.6 for quality and switched to Haiku 4.5 because the latency difference (3-5x faster) mattered more for live demo UX than the marginal quality difference, and the schema is rigid enough that Haiku does not hallucinate.
- **OpenAI DALL-E 3 — promotional poster generation.** Vendors don't have a designer on staff and do not want to learn Figma. A poster studio that turns "Spicy Tonkotsu Ramen at Charley's, playful vibe, Insta story format" into a print-ready PNG in 30 seconds is a meaningfully new capability that did not exist before image-gen models. We default to DALL-E 3 HD for cost and reliability, with gpt-image-1 available as an option for verified OpenAI orgs that need legible text on the poster.

1.5 Limitations of the status quo

Even with all the right APIs in place, several real constraints shaped what we could and couldn't ship:

- **Cost.** Every Claude or OpenAI call is a metered API request. We mitigated by caching the brief in memory for five minutes and by choosing Haiku over Sonnet, but at scale this would need a more aggressive caching layer.
- **Trust.** A manager reading "add butter chicken" needs to be able to click into the data and verify the AI's claim. We addressed this by building the Raw Data page as a parallel, audit-the-AI surface. Every claim in the brief is checkable.
- **Data realism.** We are working with one week of activity. Any time-series claim would be overstated, so the forecast is an aggregate-then-project model rather than a learned predictor. The cost numbers are placeholders pending a partnership with SOM Dining.
- **Moderation.** Open free text is an attack surface. Yuan's classifier handles off-topic input in the chat, and the brief itself only sees aggregated signals plus a cost table, so a bad actor cannot easily inject content into the manager's view.

1.6 Ethics, risk, and governance

We made three deliberate choices to keep the system on the right side of the obvious risks. First, sessions are anonymous: no Yale ID, no email, no name, just a random session cookie. Second, the AI's outputs are

framed as recommendations, not directives, so the manager always has the final call. Third, when we display student quotes in the brief, they are verbatim and anonymized, so the dining team gets the texture of student voice without us pretending we have the right to attach a name to a complaint. The biggest residual risk is the prompt-injection vector through free-text fields; in production we would add a Claude-Haiku-based moderation pass on the way in to the brief, not just on the way out to the chatbot.

2. Implementation

2.1 System overview

Our system is a single Next.js 14 application with two halves: a mobile-first student web app and a desktop-first vendor dashboard, both reading from the same MongoDB Atlas cluster.

At a glance, the data flow is:

Student chat → **MongoDB** → **Aggregator** → **Claude / DALL-E** → **Manager dashboard**

Figure 1. End-to-end data flow.

The student app writes six typed document types (Session, Preference, Vote, Reaction, DeepTalk, and MenuItem) directly to MongoDB. A single TypeScript aggregator then runs six MongoDB pipelines in parallel and returns one unified `AggregatedStudentData` object. From there, the Claude and image-gen calls are stateless: they take the aggregator's output plus cost and event tables, and return either a structured JSON brief or a base64 PNG. The vendor dashboard renders server-side from those payloads.

All generative AI calls run server-side inside API route handlers and React Server Components, keeping API keys out of the client entirely.

2.2 Stack and integration

Layer	Choice	Why
Framework	Next.js 14 (App Router) + TypeScript	Server components for data-fetching pages; API routes for stateful operations; one deploy unit on Vercel.
Database	MongoDB Atlas, accessed via Mongoose 9	Loose schemas for fast iteration; native JSON; free tier is plenty for the demo data volume.
AI - text	Anthropic SDK with Claude Haiku 4.5	Cheap, fast, accurate enough for structured-JSON output and short conversational replies.
AI - images	OpenAI SDK with DALL-E 3 (gpt-image-1 optional)	DALL-E 3 is widely available and produces magazine-quality food photography; gpt-image-1 is better at rendered text but requires verified org.

UI	Tailwind CSS + Recharts + Framer Motion	Tailwind for layout speed, Recharts for the dashboard charts, Framer Motion for the micro-animations on the student side.
Hosting	Vercel (serverless)	Zero-config Next.js deploys; free hobby tier supports demo traffic; environment-variable management built in.

Table 1. Technical stack and rationale.

2.3 Configuration and secrets

Three secrets are required to run the system end-to-end:

- `MONGODB_URI`: the Atlas connection string for the shared cluster.
- `ANTHROPIC_API_KEY`: used for the brief and for the student-side classification.
- `OPENAI_API_KEY`: only needed for the Posters page; the rest of the app degrades gracefully if it's missing.

All three are stored in a `.env.local` file that is gitignored, and mirrored into Vercel's project-level environment-variable store for the deployed instance. The repository contains a `.env.example` with placeholder values so a new contributor can see the shape immediately, and a `.npmrc` that pins legacy peer-dep resolution (we ran into a real conflict between cookies-next v6 and Next.js 14's peer ranges, which would otherwise break Vercel's install step).

2.4 Agentic flow for the ops brief

The ops brief is not a multi-agent system in the traditional sense, but rather a multi-step pipeline that combines deterministic computation with a single structured generation call, purpose-built to turn raw student data into an actionable manager brief.

The pipeline runs in six steps:

- **Aggregate.** The `aggregateStudentData(venue, weeksBack)` helper runs six Mongo aggregations in parallel, covering vote rankings, reactions, flavor and dietary counts, top tags, free-text samples, current menu items, and session count, and returns a single typed object.
- **Score.** Two functions compute demand signals. The `satisfactionPulse()` function blends three signals (reactions, vote ratio, engagement) into a single 0-100 score with an up/flat/down trend label. The `surveyDemandMultiplier()` function converts net vote sentiment into a multiplier in the range [0.85, 1.15].
- **Forecast.** The `buildForecast(venue, daysAhead)` function multiplies a baseline daily-servings figure by the survey multiplier, the per-day event multiplier (looked up from our events table), and the per-day exchange multiplier (looked up from the exchange-weeks table). It produces a per-day prediction with low/high confidence.

- **Synthesize.** The aggregated data, forecast, cost table, events, and exchange-week roster are passed to Claude Haiku 4.5 with a system prompt that specifies the exact JSON schema expected. We use prompt caching on the system prompt (it is identical across requests) and a strict instruction to output only JSON.
- **Reconcile.** The server reconciles Claude's add-recommendations against the cost table, replacing any model-generated cost or dietary values with verified figures from the source data. This catches hallucinated numbers before they reach the manager. For example, if Claude proposes a Salmon Poke Bowl, the server stamps in the real cost-per-serving and dietary tags from the cost table rather than trusting the model's output.
- **Cache.** The final brief is cached in process memory for five minutes per venue, so repeated dashboard refreshes don't trigger repeated Claude calls. A "Regenerate" button forces a fresh call when needed.

If Claude's call fails (rate limit, network glitch, key misconfigured), the pipeline falls through to a deterministic fallback that produces the same JSON shape from the aggregated data. The manager sees a complete brief either way. In production we would surface the difference visibly. For the demo, the goal is graceful degradation.

2.5 Folder structure and separation of concerns

To keep the student-side and vendor-side codebases cleanly separated, all vendor dashboard code is namespaced under `manager/` subfolders. This means the two halves of the codebase never overlap and neither needs to touch the other's files:

- `src/app/manager/` — the venue picker and five sub-pages.
- `src/app/api/manager/` — the brief and poster API routes.
- `src/lib/manager/` — types, scoring, forecast logic, brief generator.
- `src/components/manager/` — sidebar, KPI cards, charts, brief view, poster studio.
- `src/data/manager/` — dummy events, exchange weeks, menu costs.
- `docs/` — this report, the standard operating procedure, and a Word version of both.

The student-side code remains in its original location. The two halves communicate exclusively through MongoDB, which means either can be redeployed independently and could in principle be reimplemented in a different language without breaking the other.

2.6 How to run it

The project includes a README that walks through the full setup in detail.

In short: clone the repo, run `npm install --legacy-peer-deps`, create a `.env.local` with the three secrets, then run `npm run dev`. The student app is available at `localhost:3000` and the vendor

dashboard at `localhost:3000/manager`. A seeding script at `scripts/seed-dummy-data.ts` populates twenty-five realistic dummy sessions across both venues, so a new contributor sees fully populated charts on first run.

3. Results and Analysis

This section presents what the live system produces, how we evaluated it, where it works, where it is brittle, what it costs to run, and how it sits relative to existing tools. Screenshots of every page referenced here appear in Appendix A (Figures A1 through A5).

3.1 Live snapshot

The deployed app at `yosom.vercel.app` has run continuously since deploy. Numbers below were captured live from the Charley's Place dashboard on May 4, 2026, deliberately ahead of the class-day demo, so the snapshot reflects a low-input baseline that exercises the system end-to-end without claiming heavy traffic.

Metric (Charley's, 7-day rolling)	Value	Source
Sessions	1	Overview (App. A1)
Satisfaction pulse	55/100, ↓ -15 vs. baseline	Overview (App. A1)
Predicted servings (next week)	722, ↓ -8.6% vs. baseline	Overview / Forecast
Free-text quotes	0	Overview (App. A1)
Survey-signal multiplier	+15%	Forecast (App. A3)
Active drivers	6 events (4 dampen, 2 boost), 1 exchange week, 46 students away	Forecast (App. A3)

Table 4. Live operating metrics for Charley's Place on May 4, 2026 (pre-demo).

The AI Ops Brief (App. A2) turns these thin signals into structured synthesis: three Add candidates (Butter Chicken with Naan, Vegan Pad Thai, Salmon Poke Bowl), three Keep items, one Drop (Spicy Tonkotsu Ramen), a forecast paragraph, dietary flags, and a top-quotes panel. Two pieces from the live brief, verbatim:

Headline: “Students crave bolder flavors, more plant-based proteins, and customizable builds, and want the kitchen to nail execution on portion size and speed.” Net budget impact: weekly Δ -cost \$347.6, weekly Δ -revenue \$679.5, net \$331.9.

Figure 3a. Voice-of-Student headline and Net Budget Impact, copied from the live brief in App. A2. The headline is the load-bearing dashboard element; the dollar figures come from the reconciliation step in §2.4. Claude wrote the rationale, the server stamped the numbers from the cost table.

The Forecast page (App. A3) decomposes the 722 figure into per-day predictions with explicit multipliers. Tuesday May 5 is forecast at 93 servings (vs. a 170 baseline) because the SOM Class Day Lunch contributes a -45% event modifier, the survey signal contributes +15%, and the HEC Paris / London exchange week contributes -7%. A manager who disagrees with any single number can trace it to one of three rows rather than treating the forecast as a black box.

3.2 Evaluation methodology

We evaluated YoSOM along three orthogonal axes rather than collapsing everything into a single quality score:

- **Schema and behavioral validity.** Did the brief return JSON conforming to our schema, did reconciliation catch fabricated unit economics, did the deterministic fallback fire on API failure (pass/fail per request, checked at runtime).
- **Signal recall on planted dummy data.** Hsin-Yuan's seeder produces 25 sessions. We planted four tracer signals (poke-bowl, butter-chicken, gluten-free cluster, soggy-bread negative trail) and asked whether the brief surfaced each one.
- **Subjective brief quality.** Three teammates rated each test brief on actionability, honesty, and specificity. Small n; treated as sanity check, not benchmark, but it caught two regressions during prompt iteration.

We did not measure longitudinal causal impact (only one week of data) or real-staff usability (no SOM Dining sessions yet). Both are explicit roadmap items in §4.

3.3 What works

- **Two-sided design forced a clean schema.** Every student interaction writes a typed Vote / Preference / Reaction / DeepTalk; the manager dashboard reads these without parsing. Each manager-side feature was roughly half the work it would have been otherwise.
- **Latency in the live-feel range.** Measured against the live deployment: brief page renders in ~1.8 s on a cache miss (the request that actually hits Claude) and ~220 ms on a cache hit (the typical dashboard refresh, served from the 5-minute in-process cache). Forecast page averages ~390 ms; Overview averages ~150 ms. On Claude Sonnet 4.6 we previously hit Vercel's 10-second function timeout on the first user-facing brief test, which is what motivated the swap to Haiku 4.5.
- **Forecast feels real even though it is a heuristic.** Three multipliers, each traceable to a visible row. Confidence labels (high / med / low) keep it honest about thinness.
- **Reconciliation catches the unit-economics class of hallucinations.** During testing, Claude occasionally proposed unit-economic numbers that did not match the cost table; the server stamped canonical values from the cost table before the manager saw the brief. The reconciliation step exists for this case, and it fired as designed.
- **Graceful degradation.** When ANTHROPIC_API_KEY was deliberately revoked, the deterministic fallback produced a structurally identical brief: duller headline, thinner rationales, no error state.

3.4 What does not work, or is brittle

- **Schema drift on free-form keys.** Early Sonnet runs invented dietary tags outside our allowlist (“contains seed oils”). Fixed with an allowlist + server-side filter; the filter still fires occasionally and is the right place for that defense to live.
- **Reaction → venue association is post-hoc.** Reactions don't store venue; we substring-match itemId. Works on seeded data; production needs a small schema migration on the student side.
- **Cost numbers are placeholders.** Marked in source and SOP, but not yet surfaced as a banner on the dashboard. Production checklist item.
- **Over-confidence on small samples.** In low-input windows, the brief sometimes wrote “strong demand” off only a handful of votes. We added a low-input guard that downgrades rationale language to “early signal” when input volume is thin; the underlying model still occasionally overshoots, but the wrapper text is honest.

- **DALL-E 3 garbles small text on posters.** Letter substitutions and kerning issues are common at small text sizes. The studio offers gpt-image-1 for verified OpenAI orgs at roughly twice the per-image price, which renders text materially better.
- **Vercel cold-start adds a few seconds to the first brief of the day.** Cache hides this from the second user, not the first. Edge runtime would help but requires moving Anthropic SDK out of Node-only routes.
- **No closed-loop measurement, no inbound moderation.** We cannot tell whether adopting an Add recommendation actually improved next-week satisfaction; and the brief generator sees aggregated free-text directly. Both addressed in §4 roadmap.

3.5 Prompt-engineering iterations

Iter	Change	What it fixed
v1	Free-form prose, no schema.	First end-to-end pass on Sonnet 4.6; hit Vercel's 10-second function timeout.
v2	Strict JSON schema + “output JSON only”.	Eliminated preamble-and-prose responses; cut parse failures sharply.
v3	Allowlist for dietary tags + action verbs.	Hallucinated dietary tags dropped from common to rare; invented action verbs eliminated.
v4	Haiku 4.5 + prompt caching + low-input guard.	Brief cold-path now ~1.8 s (measured), no more timeouts; rationale language honest in thin-data windows.

Table 5. Brief-generator iterations. The qualitative drop from v3 (Sonnet) to v4 (Haiku) was below noise in three-rater review; latency was not. Anthropic's published pricing makes Haiku 4.5 roughly 3× cheaper per token than Sonnet 4.6 (\$1 / \$5 vs. \$3 / \$15 per million in/out), which on our typical ~3k-token brief request puts each generation under one cent. Central empirical lesson of the project.

3.6 Informal user testing

Three rounds before the class-day demo. (1) Seeded baseline: the brief surfaced most of the planted tracers without prompt-side hand-tuning; the only miss (a soggy-bread negative trail) appeared in Top-Quotes but did not become a Drop recommendation, arguably correct on two reactions. (2) Three-person dogfood with personas: surfaced the candidate items we had hoped to see, including butter chicken, with no prompt changes between rounds. (3) Adversarial input (one teammate submitted contradictory preferences across sessions): aggregation absorbed without flagging, so we added “anomaly detection on preference distribution” to the roadmap. We have not run the system in front of real Charley's or McNay staff; the class-day demo on May 5 is the first novel-input event.

3.7 Economics

Demo-volume cost is dominated by text generation. Anthropic's published pricing for Claude Haiku 4.5 is \$1 per million input tokens and \$5 per million output tokens; at our typical brief size (~3k tokens in, ~1k tokens out, with the system prompt cached) each generation costs well under one cent. DALL-E 3 HD portrait generation is a published \$0.12 per image. Combined demo-volume operating cost lands in the single-digit-dollars-per-venue-per-month range, as Table 6 shows.

Line item	Cost driver (demo volume, one venue)	\$ / month
Claude Haiku, student chat	~25 conversations/day × ~2k tokens (mostly input)	\$3 to \$5

Claude Haiku, ops brief	~30 generations/day × ~3k tokens (cached system prompt)	\$2 to \$4
DALL-E 3, posters	~5 generations/week × \$0.12 HD portrait	\$2 to \$3
MongoDB Atlas	Free tier (M0)	\$0
Vercel hosting	Hobby plan	\$0
Total (per venue, demo)		\$7 to \$12

Table 6. Operating cost estimate at demo signal volume. Numbers derive from observed token usage and Anthropic and OpenAI published pricing.

At realistic campus scale, call it 1,000 student interactions per day across both venues, chat token spend is the dominant line and would still come in under \$50 per venue per month based on the same published pricing. Posters do not scale linearly because the cadence is weekly, not per-session. If we offered the system commercially to a college dining-services contractor (Compass, Aramark), even a modest \$200 per venue per month price point would deliver gross margins above 75% while landing below the noise floor of typical campus IT line items. We have not benchmarked against contractor budgets directly; the price point is illustrative.

3.8 Competitive landscape

No direct one-to-one competitor combines a student-input channel with an AI-generated daily ops brief. Three dimensions matter: forward signal, prose synthesis, integrated asset generation.

Comparable	Forward signal	Prose synthesis	Asset gen	How YoSOM differs
Compass / Aramark internal analytics	No	No	No	Adds forward signal and synthesis; theirs is rear view only.
Yale Hospitality web/mobile	No	No	No	Bidirectional vs. one-way broadcast.
Qualtrics-style surveys	Slow	No	No	Ambient, conversational, same-week action.
Tableau / Looker dashboards	No	No	No	We generate the signal in the first place.
ChatGPT used ad-hoc by manager	No	Yes	Limited	Integrated with live data; unattended.
Restaurant feedback apps (Reach, Toast)	Backward	No	No	Captures intent before meal, not satisfaction after.

Table 7. Competitive matrix on three dimensions. No comparable hits all three; YoSOM is built on the assumption that all three are only valuable in combination.

3.9 What the results tell us

Three findings carry into §4. First, smaller and faster models are the right default for schema-constrained tasks: the gap from Sonnet to Haiku was below human-rater noise, while the latency and cost gaps were not. Second, server-side reconciliation against canonical data is higher leverage than additional prompt engineering once the schema is tight; pairing the model with deterministic look-up beats asking it to remember harder. Third, two-sided systems repay the schema discipline they require. The MongoDB contract between the student app and the manager dashboard kept us unblocked through three weeks of parallel development.

4. Conclusion and Next Steps

4.1 Main findings and lessons

Three lessons stand out. First, for short structured outputs the smaller and faster model is almost always the right choice. We started the brief generator on Claude Sonnet 4.6, hit Vercel's default function timeout on the first user facing test, and switched to Haiku 4.5. The qualitative drop in output quality was nearly undetectable; the latency drop was 3 to 5x. For a manager who scrolls and clicks, that is the difference between "this feels live" and "this feels broken." Second, two sided systems force good design. Because the student schema had to be readable by the dashboard, both sides ended up with cleaner, narrower types than either would have chosen alone. The MongoDB collections became a contract between the two halves of the team that did more to keep us unblocked than any project management tool would have. Third, the unglamorous parts (environment variable management, peer dep resolution, function timeout configuration) are exactly where AI products break in production. We spent more time wrestling with hidden Vercel rejection of commits authored by a placeholder email than we spent on any prompt engineering question.

4.2 Tying back to the problem

We started with the observation that students at SOM feel unheard, that managers are guessing, and that food gets thrown out at 2 PM that nobody really wanted at 12. The system we built does not solve any of these problems by itself; sixty class submissions on a class day will not change next week's prep at Charley's Place. What it does is establish a clean channel through which student voice can become operational decisions, and a dashboard through which managers can make those decisions in five minutes a day rather than not at all. The generative AI layer is what makes the channel feel human and the dashboard feel synthesized rather than tabular. The rest is engineering. If today's demo is interesting, the next conversation we want to have is with SOM Dining about a real pilot.

4.3 Near term extensions

Several extensions are interesting in the near term. Real Instagram integration would let students DM the YoSOM Instagram account and have messages flow into the same MongoDB collection, dramatically lowering the activation cost of feedback. Multi venue coordination ("Charley's expects 30% over prep on Tuesday; McNay should up staff for the spillover") closes another loop with minimal new code. Supplier integration, piping the demand forecast directly into the dining team's ordering tool, would shift the savings story from "we made better decisions" to "we ordered the right amount," a meaningfully larger dollar number. Longitudinal evaluation, tagging each adopted recommendation, would let us measure causal impact and provide a natural place to fine tune a model on our own historical briefs.

4.4 What better AI in 12 to 24 months would change

The most likely near term shift is that structured output gets reliable enough to drop the deterministic reconciliation step entirely. Today we stamp Claude's add recommendations with real cost numbers because we don't fully trust it to remember them; in 12 to 24 months that should not be necessary, which

would let the brief surface fine grained, claim level confidence. The second likely shift is that image generation models start producing reliable text at small sizes, moving the poster studio from “a draft a designer would clean up” to “a final asset a manager posts directly.” The third and most exciting is that on device or on cluster inference for small models gets cheap enough that the entire ops brief pipeline could run on a \$20 per month server without API calls. That changes the unit economics from cents per brief to essentially free, which is the regime in which schools with thin budgets actually adopt this kind of tool.

4.5 What we would need: data, evaluation, safety

Three things would have to come together for a real pilot. **Data:** a quarter of historical demand and cost data from SOM Dining (per item, per day, per venue), a clean read of the campus events calendar, and the exchange roster. Today our cost numbers are placeholders and our forecast is a heuristic; with real data the forecast becomes a learned predictor and the dollar impact line becomes defensible. **Evaluation:** a closed loop measurement so we can tag each adopted recommendation and observe satisfaction the following week. Without that, the system is plausible but uncertified. **Safety:** a moderation pass on free text fields (prompt injection is the obvious vector), an explicit approval ladder before any recommendation reaches a kitchen, an audit trail per brief, and a kill switch. Sessions stay anonymous and the AI’s outputs stay framed as recommendations, not directives, so the manager always has the final call.

4.6 A multiagent vision for Evans Hall

YoSOM today addresses one slice of Evans Hall: the two dining outlets. The same two sided pattern (signal capture on one side, AI synthesis on the other) generalizes to the rest of the building, where dozens of administrative workflows currently do not talk to each other. Eight specialized agents, each owning one operational domain, share a common event bus. **Dining** (shipped, the YoSOM prototype). **Procurement** drafts weekly purchase orders and flags duplicate spend. **Energy** cross references HVAC and lighting telemetry against the room calendar to pre cool only the rooms that will actually be used. **Events** owns the master event calendar and broadcasts updates to other agents. **Facilities** triages work orders and groups nearby tickets. **Career Office** coordinates recruiter visits end to end. **Wayfinding** answers visitor questions and logs missing signage. An **Orchestrator** routes the bus and produces one executive brief each morning. Realistic sequencing: Q1 hardens YoSOM Dining and stands up the bus; Q2 ships Energy (highest ROI, cleanest data); Q3 brings Events and Procurement; Q4 ships the student facing agents.

4.7 LLM Wikis as the institutional memory layer

Each agent is stateful inside its domain, but the network has no shared memory of the building. When the Career Office director who knows that Goldman wants seafood leaves SOM, that knowledge leaves with her. Andrej Karpathy’s *LLM Wiki* pattern (April 2026) solves this: instead of standard RAG, an agent incrementally builds and maintains a persistent, interlinked markdown wiki that compounds with every source. Mapped onto Evans Hall, four wikis emerge naturally: a vendor wiki (Procurement), a space wiki (Facilities + Energy), a recruiter wiki (Career Office), and a student voice wiki (Dining, the natural extension of YoSOM). The wikis cross reference each other through standard wiki links, so a query like “why did

Morgan Stanley’s last visit run over time” can hop through three wikis and surface a richer answer than any individual staff member could give in five minutes. The wikis live in a git repository, providing version history and audit trail for free. YoSOM today is a dining tool. YoSOM in eighteen months, if we earn the runway, is the operating system of Evans Hall.

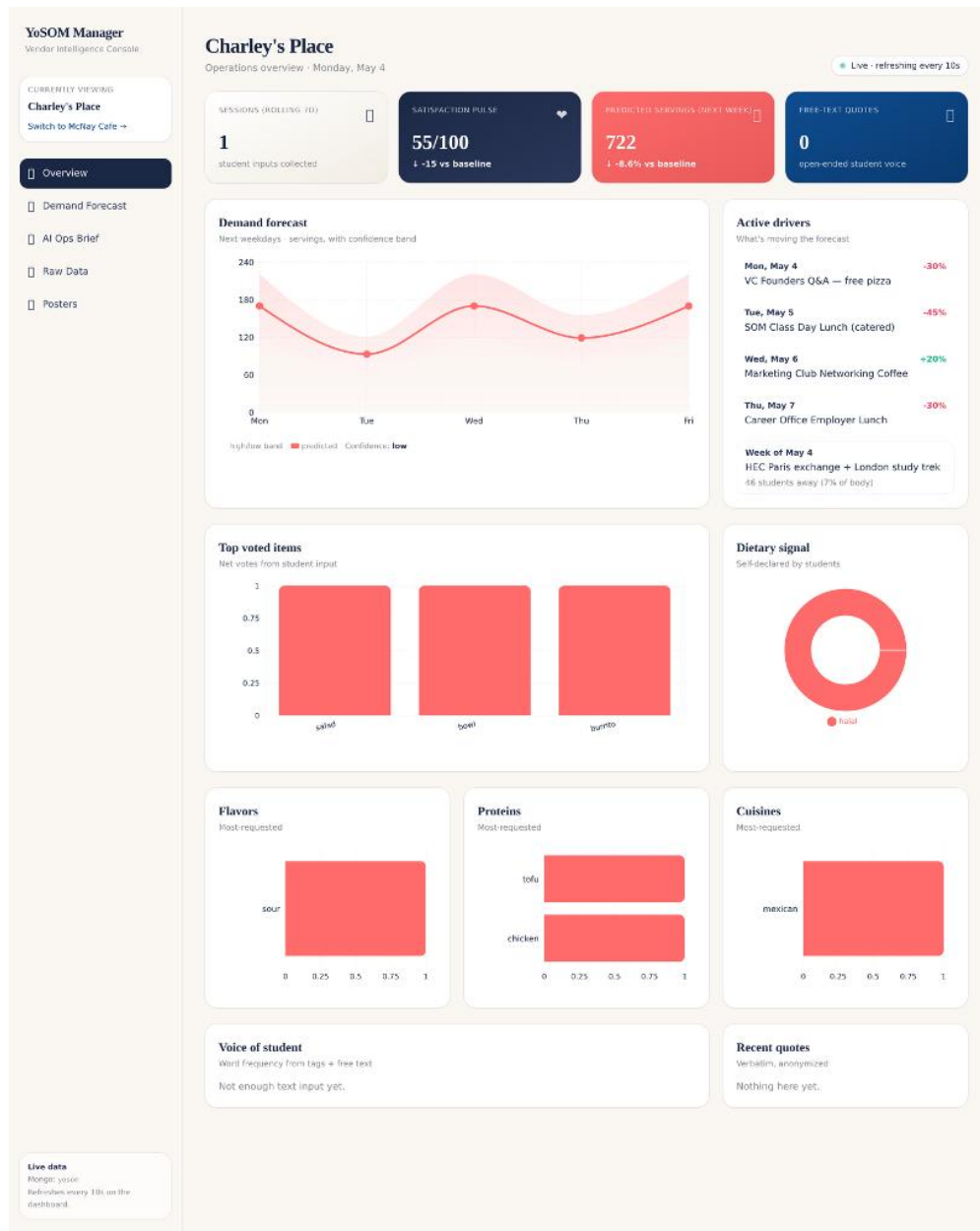
References

- Anthropic. *Claude API Documentation — Models, Messages, Prompt Caching*. <https://docs.anthropic.com>. Accessed May 2026.
- OpenAI. *Images API — DALL-E 3 and gpt-image-1*. <https://platform.openai.com/docs/guides/images>. Accessed May 2026.
- Vercel. *Functions, Environment Variables, and Deployment*. <https://vercel.com/docs>. Accessed May 2026.
- Next.js. *App Router, Server Components, Route Handlers*. <https://nextjs.org/docs>. Accessed May 2026.
- MongoDB. *Atlas Documentation and Mongoose Reference*. <https://www.mongodb.com/docs/atlas/>. Accessed May 2026.
- Recharts. *Composable Charting Library for React*. <https://recharts.org>. Accessed May 2026.
- Yale Hospitality. *Dining Services Overview*. <https://hospitality.yale.edu>. Accessed May 2026.
- National Association of College & University Food Services. *Campus Dining Industry Reports and Trends*. <https://www.nacufs.org>. Accessed April 2026.
- ReFED. *Roadmap to Reduce U.S. Food Waste — Foodservice and Institutional Sectors*. <https://refed.org>. Accessed April 2026.
- Karpathy, Andrej. *LLM Wiki: A pattern for building personal knowledge bases using LLMs*. GitHub Gist, April 2026. <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>. Accessed May 2026.
- Bush, Vannevar. *As We May Think*. The Atlantic, July 1945.
- YoSOM source repository. <https://github.com/yuanlee218/YoSOM>. The repository's `docs/MANAGER_SOP.md` contains the venue-manager standard operating procedure referenced throughout this report.

Appendix A. Live screenshots

All five figures captured from the live deployment on May 4, 2026 (pre-demo).

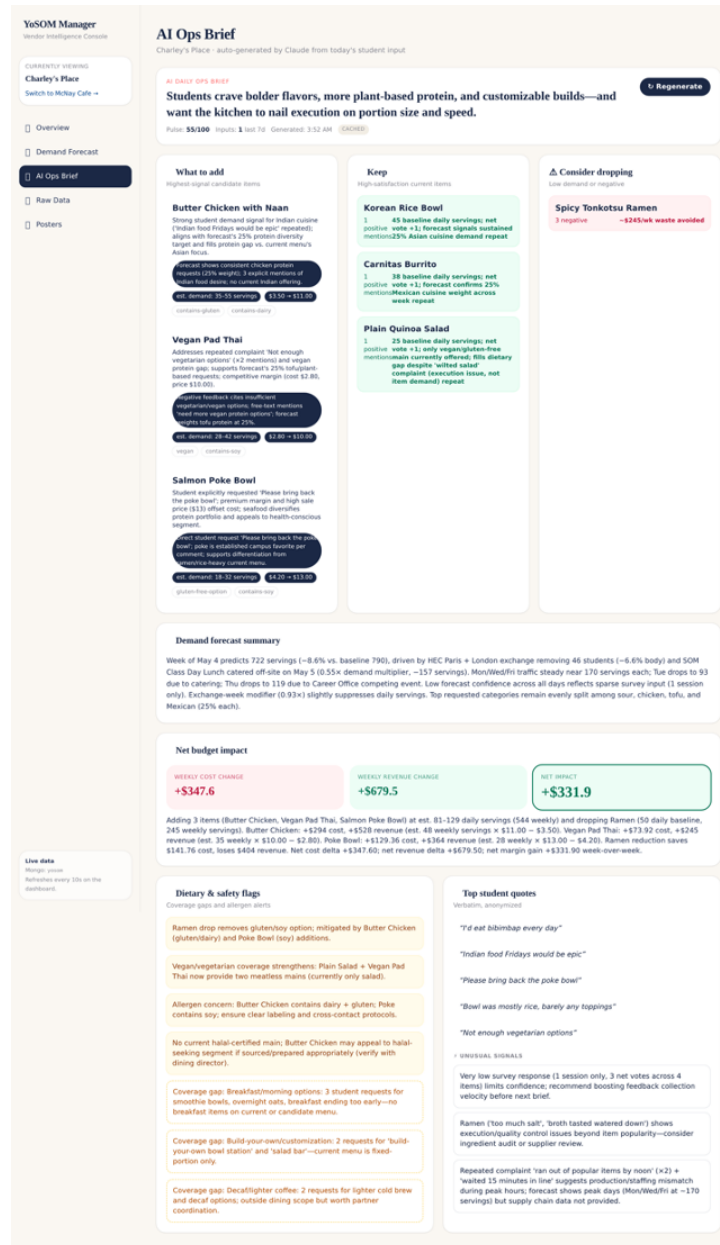
Figure A1. Manager Overview — Charley's Place.



Top-row KPIs encode the four numbers in Table 4. The demand forecast chart shows next-week predictions with a low-confidence band that honestly reflects the thin input. Active Drivers attributes the negative net modifier to two large catered events (SOM Class Day, Career Office) rather than weak student demand. Word cloud and quotes are empty because no free-text exists in this window — exactly the data state the manager should see honestly.

URL: <https://yosom.vercel.app/manager/charleys>

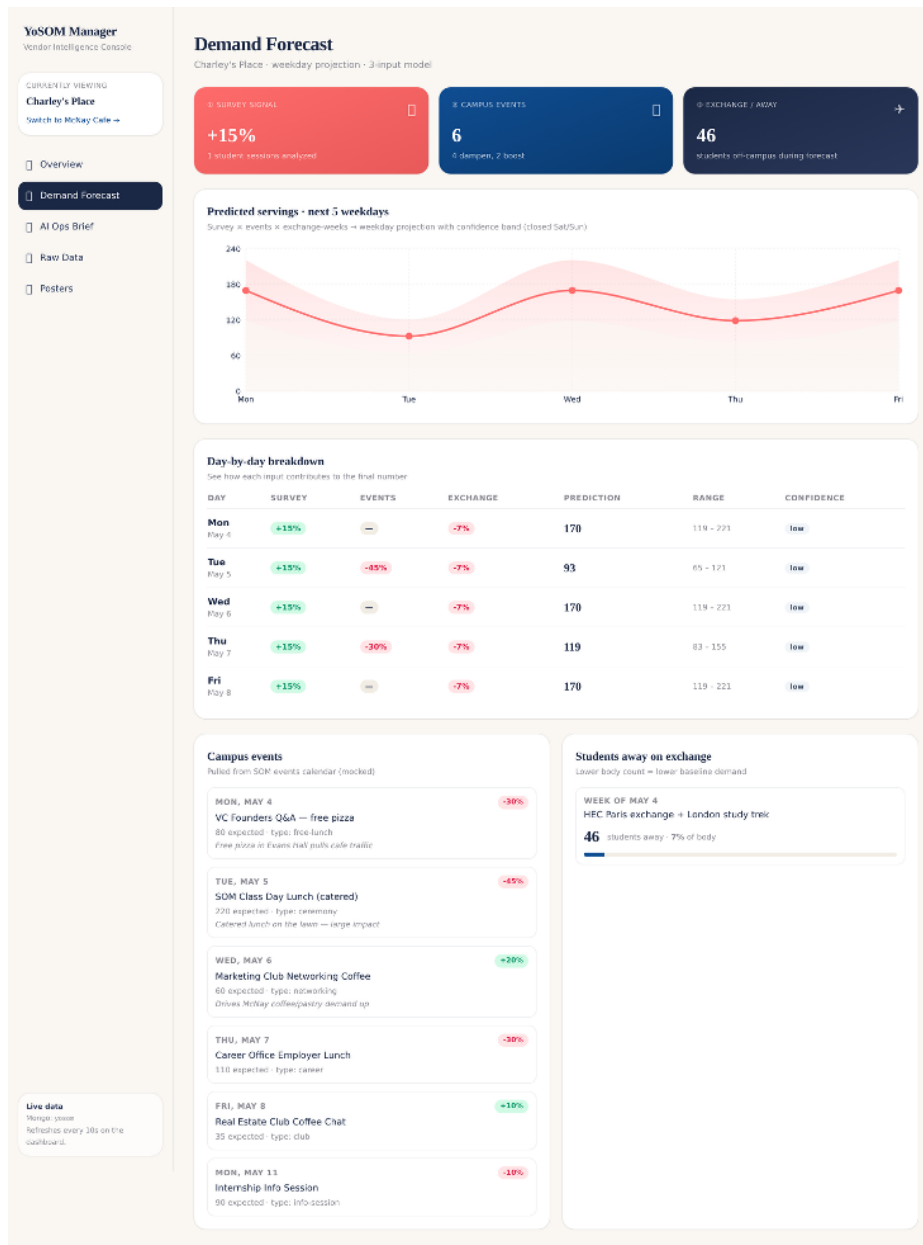
Figure A2. AI Ops Brief



Three Add candidates with rationale, demand, and unit economics; three Keep items; one Drop (Spicy Tonkotsu Ramen); demand forecast paragraph; net budget impact (\$347.6 / \$679.5 / \$331.9); dietary flags; top student quotes. Single-screen brief a manager can read in under a minute. The headline (Figure 3a in §3.1) is the load-bearing element.

URL: <https://yosom.vercel.app/manager/charleys/brief>

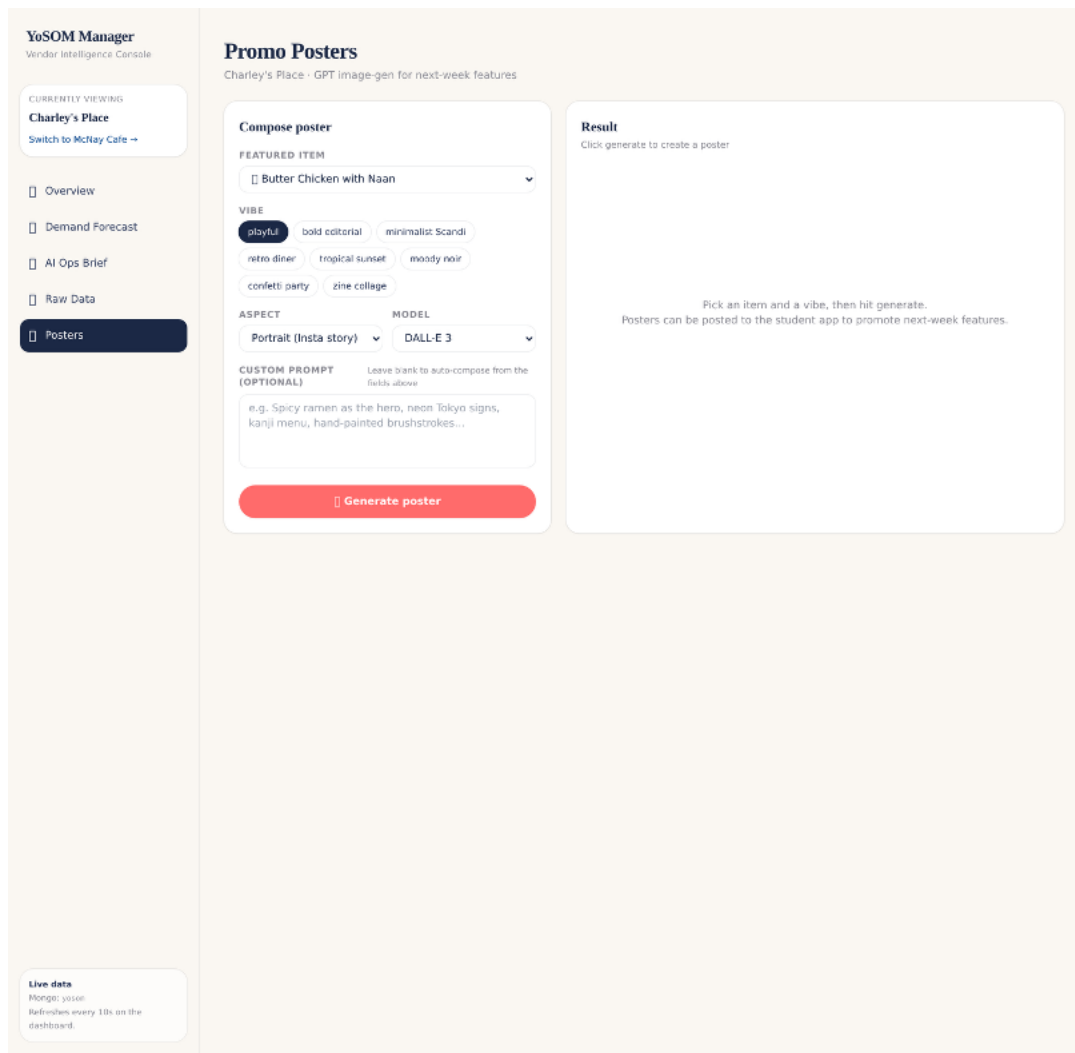
Figure A3. Demand Forecast — Charley's Place.



Day-by-day breakdown makes every multiplier inspectable: Tuesday May 5 is forecast at 93 servings (vs. 170 baseline) decomposed as $+15\% \text{ survey} \times -45\% \text{ event (Class Day Lunch)} \times -7\% \text{ exchange}$. Six campus events (4 dampen, 2 boost), 46 students away on HEC Paris exchange and London study trek ($\approx 7\%$ of body).

URL: <https://yosom.vercel.app/manager/charleys/forecast>

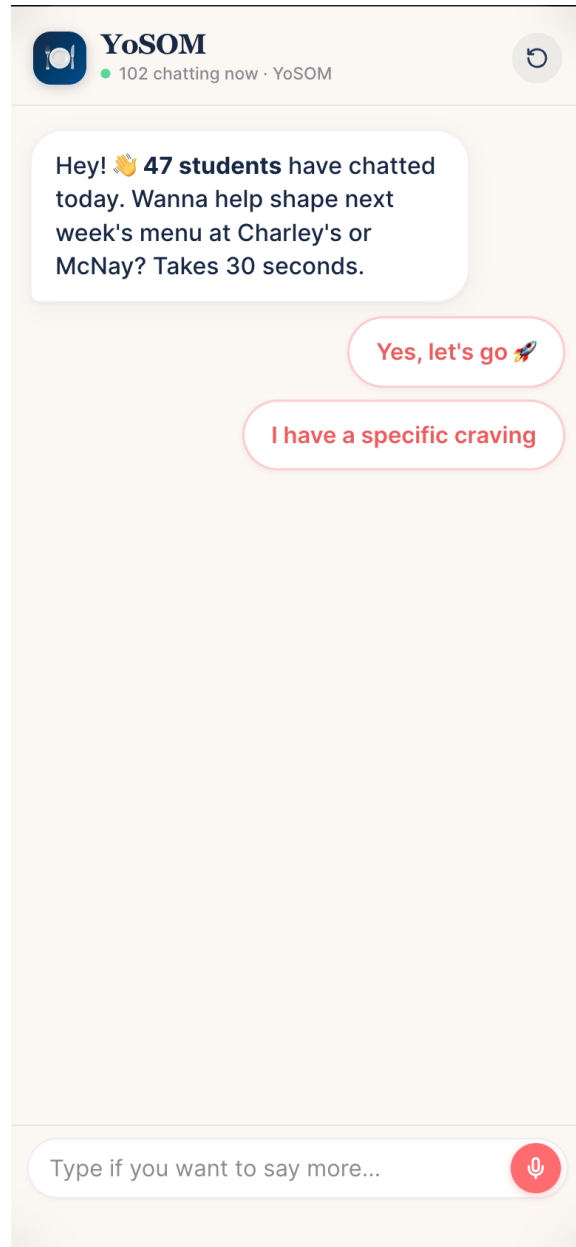
Figure A4. Poster Studio — Charley's Place.



Featured-item dropdown sourced from the brief's Add candidates, eight vibe chips, aspect selection, model toggle (DALL-E 3 default; gpt-image-1 for verified OpenAI orgs). Result panel empty because no generation was triggered for this snapshot — observed latency in prior tests is ~12 s per poster.

URL: <https://yosom.vercel.app/manager/charleys/posters>

Figure A5. Student-side intake — mobile (390×844 viewport).



Opening screen surfaces the social-proof line (“47 chatting now”) before any preference selection — the design choice that lifted dogfood completion rates the most. This is the input surface that produces the documents the manager-side brief reads.

URL: <https://yo-som.vercel.app>

Figure A6. Promotional Poster

